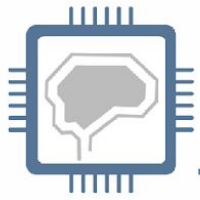


***Inteligência Artificial
e Computacional***

Disciplina: Linguagens de Programação para Inteligência Artificial

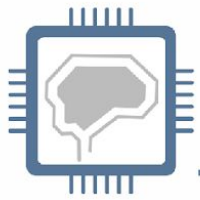
Aula 5: Classes

Profa.: Sabrina de A. Silveira



Classes

- No contexto da programação orientada a objetos, as classes representam coisas e situações do mundo real.
- Com base nas classes, criamos ou “instanciamos” objetos.
- A classe funciona como um "molde" para a criação de objetos. Uma classe define um comportamento geral que uma categoria de objetos deve apresentar.
- Nesta disciplina veremos alguns conceitos elementares sobre as classes.



Classes 1 (creating)

A classe Dog será criada para representar o animal cachorro de modo geral.

Essa classe conterá dados sobre o cachorro, **name** e **age**, bem como comportamentos que ele pode apresentar, como **sit** e **roll_over**.

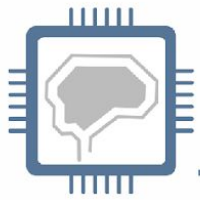
A classe Dog foi armazenada dentro de um arquivo de nome dog.py

```
class Dog:
    """modelagem do animal cachorro"""

    def __init__(self, name, age):
        """inicializa os atributos name e age"""
        self.name = name
        self.age = age

    def sit(self):
        """Simula o cachorro atendendo ao comando sentar"""
        print(f"{self.name} está sentado.")

    def roll_over(self):
        """Simula o cachorro atendendo ao comando rolar"""
        print(f"{self.name} rolou!")
```



Classes 1 (creating)

O nome da classe começa com **class** Dog:
letra maiúscula em Python, por definição.

Na segunda linha, temos um docstring que descreve a função da classe: `"""modelagem do animal cachorro"""`

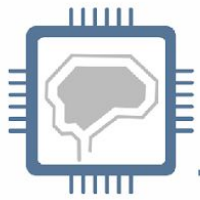
As **funções** dentro da classe são chamadas de **métodos**.

```
class Dog:
    """modelagem do animal cachorro"""

    def __init__(self, name, age):
        """inicializa os atributos name e age"""
        self.name = name
        self.age = age

    def sit(self):
        """Simula o cachorro atendendo ao comando sentar"""
        print(f"{self.name} está sentado.")

    def roll_over(self):
        """Simula o cachorro atendendo ao comando rolar"""
        print(f"{self.name} rolou!")
```



Classes 1 (creating)

O método `__init__` é chamado automaticamente quando usamos a classe.

Esse método, por convenção, deve ter 2 caracteres underscore no início e 2 no fim.

Se o nome não obedecer a essa convenção, o método não é chamado automaticamente quando a classe é usada.

```
class Dog:
```

```
    """modelagem do animal cachorro"""
```



```
def __init__(self, name, age):
```

```
    """inicializa os atributos name e age"""
```

```
    self.name = name
```

```
    self.age = age
```

```
def sit(self):
```

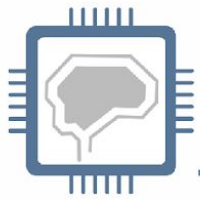
```
    """Simula o cachorro atendendo ao comando sentar"""
```

```
    print(f"{self.name} está sentado.")
```

```
def roll_over(self):
```

```
    """Simula o cachorro atendendo ao comando rolar"""
```

```
    print(f"{self.name} rolou!")
```



Classes 1 (creating)

O método

`__init__(self, name, age)`

possui 3 parâmetros:

O parâmetro **self** é necessário e precisa ser o primeiro. Ele é uma referência para a própria instância da classe.

Os parâmetros **name** e **age** precisam ser informados quando criamos uma instância da classe Dog.

```
class Dog:
```

```
    """modelagem do animal cachorro"""
```



```
def __init__(self, name, age):
```

```
    """inicializa os atributos name e age"""
```

```
    self.name = name
```

```
    self.age = age
```

```
def sit(self):
```

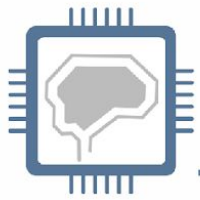
```
    """Simula o cachorro atendendo ao comando sentar"""
```

```
    print(f"{self.name} está sentado.")
```

```
def roll_over(self):
```

```
    """Simula o cachorro atendendo ao comando rolar"""
```

```
    print(f"{self.name} rolou!")
```



Classes 1 (creating)

As variáveis criadas **name** e **age** são precedidas pela palavra **self** e são os atributos da classe.

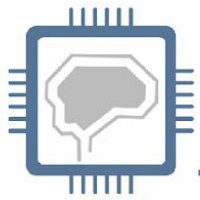
Esses atributos podem ser acessados por qualquer método da classe.

```
class Dog:
    """modelagem do animal cachorro"""

    def __init__(self, name, age):
        """inicializa os atributos name e age"""
        self.name = name
        self.age = age

    def sit(self):
        """Simula o cachorro atendendo ao comando sentar"""
        print(f"{self.name} está sentado.")

    def roll_over(self):
        """Simula o cachorro atendendo ao comando rolar"""
        print(f"{self.name} rolou!")
```



Classes 1 (creating)

A classe Dog tem ainda os métodos `sit` e `roll_over`, que não necessitam de atributos para funcionar e assim recebem apenas o parâmetro `self`.

Esses métodos apenas imprimem na tela, mas num contexto real, como um jogo, poderiam conter comandos que fariam o objeto do tipo Dog sentar e rolar.

```
class Dog:
```

```
    """modelagem do animal cachorro"""
```

```
    def __init__(self, name, age):
```

```
        """inicializa os atributos name e age"""
```

```
        self.name = name
```

```
        self.age = age
```

```
    def sit(self):
```

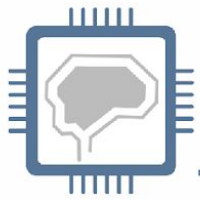
```
        """Simula o cachorro atendendo ao comando sentar"""
```

```
        print(f"{self.name} está sentado.")
```

```
    def roll_over(self):
```

```
        """Simula o cachorro atendendo ao comando rolar"""
```

```
        print(f"{self.name} rolou!")
```



Classes 1 (using)

Criaremos uma instância do tipo abstrato de dados Dog para representar um cachorro específico.

→

```
#import dog  
from dog import Dog
```

Observe a linha:

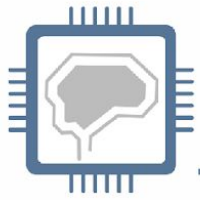
from dog import Dog

Nela importamos a classe Dog a partir do arquivo dog.py

Outra alternativa seria importar todo o módulo, opção que deixamos comentada:

import dog

```
my_dog = Dog('Willie', 6)  
print(f"O nome do meu cachorro é {my_dog.name}.")  
print(f"Meu cachorro tem {my_dog.age} anos.")  
  
my_dog.sit()  
my_dog.roll_over()
```



Classes 1 (using)

Um objeto do tipo Dog é criado com o nome Willie e idade 6.

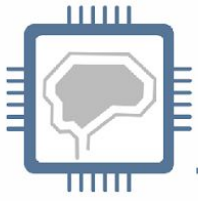
O Python utiliza o método `__init__(self, name, age)` para instanciar esse objeto.

A instância criada é associada à variável `my_dog`.

```
#import dog  
from dog import Dog
```

```
➔ my_dog = Dog('Willie', 6)  
print(f"O nome do meu cachorro é {my_dog.name}.")  
print(f"Meu cachorro tem {my_dog.age} anos.")
```

```
my_dog.sit()  
my_dog.roll_over()
```



Classes 1 (using)

Para acessar os atributos do objeto, utilizamos `my_dog.name` e `my_dog.age`

Para chamar os métodos do objeto, utilizamos o nome da variável associada a ele seguido de (.) e o nome do método, conforme `my_dog.sit()` e `my_dog.roll_over()`.

```
#import dog
```

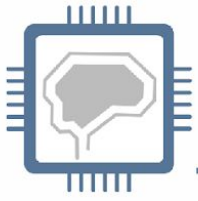
```
from dog import Dog
```

```
my_dog = Dog('Willie', 6)
```

```
print(f"O nome do meu cachorro é {my_dog.name}.")
```

```
print(f"Meu cachorro tem {my_dog.age} anos.")
```

```
{ my_dog.sit()  
  my_dog.roll_over()
```



Classes 1 (using)

```
#import dog
```

```
from dog import Dog
```

```
my_dog = Dog('Willie', 6)
```

```
print(f"O nome do meu cachorro é {my_dog.name}.")
```

```
print(f"Meu cachorro tem {my_dog.age} anos.")
```

```
my_dog.sit()
```

```
my_dog.roll_over()
```

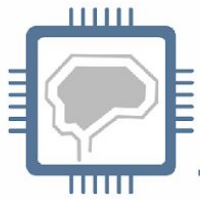
Saída do programa

O nome do meu cachorro é Willie.

Meu cachorro tem 6 anos.

Willie está sentado.

Willie rolou!



Classes 2 (creating)

A classe Car será criada para representar o objeto carro de modo geral.

Essa classe conterá dados sobre o carro, **make**, **model**, **year** e **odometer_reading**.

A classe também conterá métodos que podem descrever algum **comportamento** relacionado ao carro ou métodos para **acessar** ou **atualizar** seus atributos.

A classe Car foi armazenada dentro de um arquivo de nome car.py

```
class Car:
```

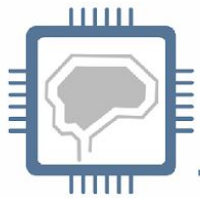
```
    """modelagem de carro através de classe"""
    def __init__(self, make, model, year):
        """inicializa os atributos que descrevem carro"""
        self.make = make
        self.model = model
        self.year = year
        self.odometer_reading = 0

    def get_descriptive_name(self):
        """Retorna um nome descritivo para carro."""
        long_name = f"{self.year} {self.make} {self.model}"
        return long_name.title()

    def read_odometer(self):
        """Imprime a distância percorrida pelo carro"""
        print(f"Esse carro rodou {self.odometer_reading} mi.")

    def update_odometer(self, mileage):
        """Atualiza valor do odômetro para o valor fornecido."""
        self.odometer_reading = mileage

    def increment_odometer(self, miles):
        """Incrementa o odômetro com o valor fornecido."""
        self.odometer_reading += miles
```



Classes 2 (creating)

O método `__init__(self, make, model, year)` possui 4 parâmetros:

O parâmetro **self** é necessário e precisa ser o primeiro. Ele é uma referência para a própria instância da classe .

Os parâmetros **make**, **model**, **year** precisam ser informados quando criamos uma instância da classe Dog.

Observe que o método `init` inicializa o atributo **odometer_reading**.

class Car:

```
"""modelagem de carro através de classe"""

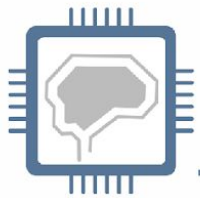
def __init__(self, make, model, year):
    """inicializa os atributos que descrevem carro"""
    self.make = make
    self.model = model
    self.year = year
    self.odometer_reading = 0

def get_descriptive_name(self):
    """Retorna um nome descritivo para carro."""
    long_name = f"{self.year} {self.make} {self.model}"
    return long_name.title()

def read_odometer(self):
    """Imprime a distância percorrida pelo carro"""
    print(f"Esse carro rodou {self.odometer_reading} mi.")

def update_odometer(self, mileage):
    """Atualiza valor do odômetro para o valor fornecido."""
    self.odometer_reading = mileage

def increment_odometer(self, miles):
    """Incrementa o odômetro com o valor fornecido."""
    self.odometer_reading += miles
```



Classes 2 (creating)

O método `get_descriptive_name` não necessita de atributos para funcionar e assim recebe apenas o parâmetro `self`.

Esse método cria uma espécie de descrição para o carro contendo ano, marca e modelo, que é armazenada na variável `long_name`.

A função `title` coloca as iniciais maiúsculas.

Note que essa função `retorna` a variável `long_name`.

```
class Car:
```

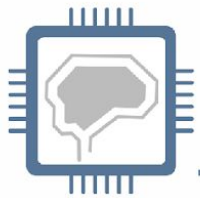
```
    """modelagem de carro através de classe"""
    def __init__(self, make, model, year):
        """inicializa os atributos que descrevem carro"""
        self.make = make
        self.model = model
        self.year = year
        self.odometer_reading = 0

    def get_descriptive_name(self):
        """Retorna um nome descritivo para carro."""
        long_name = f"{self.year} {self.make} {self.model}"
        return long_name.title()

    def read_odometer(self):
        """Imprime a distância percorrida pelo carro"""
        print(f"Esse carro rodou {self.odometer_reading} mi.")

    def update_odometer(self, mileage):
        """Atualiza valor do odômetro para o valor fornecido."""
        self.odometer_reading = mileage

    def increment_odometer(self, miles):
        """Incrementa o odômetro com o valor fornecido."""
        self.odometer_reading += miles
```



Classes 2 (creating)

O método `read_odometer` não necessita de atributos para funcionar e assim recebe apenas o parâmetro `self`.

Esse método acessa o atributo `odometer_reading` e imprime na tela uma mensagem contendo o valor dessa variável.

`class Car:`

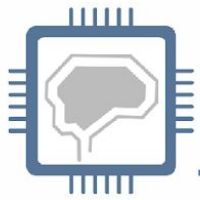
```
    """modelagem de carro através de classe"""
    def __init__(self, make, model, year):
        """inicializa os atributos que descrevem carro"""
        self.make = make
        self.model = model
        self.year = year
        self.odometer_reading = 0

    def get_descriptive_name(self):
        """Retorna um nome descritivo para carro."""
        long_name = f"{self.year} {self.make} {self.model}"
        return long_name.title()

    def read_odometer(self):
        """Imprime a distância percorrida pelo carro"""
        print(f"Esse carro rodou {self.odometer_reading} mi.")

    def update_odometer(self, mileage):
        """Atualiza valor do odômetro para o valor fornecido."""
        self.odometer_reading = mileage

    def increment_odometer(self, miles):
        """Incrementa o odômetro com o valor fornecido."""
        self.odometer_reading += miles
```



Classes 2 (creating)

O método **update_odometer** necessita receber um valor, **mileage**, que será utilizado para atualizar o atributo **odometer_reading**.

Esse método acessa o atributo **odometer_reading** e atribui a ele o valor informado no parâmetro **mileage**, **atualizando** o valor do atributo.

class Car:

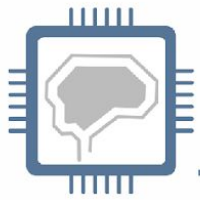
```
"""modelagem de carro através de classe"""
def __init__(self, make, model, year):
    """inicializa os atributos que descrevem carro"""
    self.make = make
    self.model = model
    self.year = year
    self.odometer_reading = 0

def get_descriptive_name(self):
    """Retorna um nome descritivo para carro."""
    long_name = f"{self.year} {self.make} {self.model}"
    return long_name.title()

def read_odometer(self):
    """Imprime a distância percorrida pelo carro"""
    print(f"Esse carro rodou {self.odometer_reading} mi.")

def update_odometer(self, mileage):
    """Atualiza valor do odômetro para o valor fornecido."""
    self.odometer_reading = mileage

def increment_odometer(self, miles):
    """Incrementa o odômetro com o valor fornecido."""
    self.odometer_reading += miles
```



Classes 2 (creating)

O método `increment_odometer` necessita receber um valor, `miles`, que será utilizado para atualizar o atributo `odometer_reading`.

Esse método acessa o atributo `odometer_reading` e atribui a ele seu próprio valor atual mais o valor no parâmetro `miles`, incrementando o valor do atributo.

`class Car:`

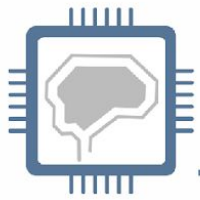
```
"""modelagem de carro através de classe"""
def __init__(self, make, model, year):
    """inicializa os atributos que descrevem carro"""
    self.make = make
    self.model = model
    self.year = year
    self.odometer_reading = 0

def get_descriptive_name(self):
    """Retorna um nome descritivo para carro."""
    long_name = f"{self.year} {self.make} {self.model}"
    return long_name.title()

def read_odometer(self):
    """Imprime a distância percorrida pelo carro"""
    print(f"Esse carro rodou {self.odometer_reading} mi.")

def update_odometer(self, mileage):
    """Atualiza valor do odômetro para o valor fornecido."""
    self.odometer_reading = mileage

def increment_odometer(self, miles):
    """Incrementa o odômetro com o valor fornecido."""
    self.odometer_reading += miles
```



Classes 2 (using)

Criaremos 2 instâncias do tipo abstrato de dados Car para representar um carro novo (**my_new_car**) e um carro usado (**my_used_car**).

Observe a linha:

from car import Car

Nela importamos a classe Car a partir do arquivo car.py



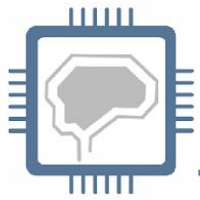
from car import Car

```
my_new_car = Car('audi', 'a4', 2019)
print(my_new_car.get_descriptive_name())
my_new_car.read_odometer()
```

```
my_new_car.odometer_reading = 23
my_new_car.read_odometer()
```

```
my_new_car.update_odometer(24)
my_new_car.read_odometer()
print()
```

```
my_used_car = Car('subaru', 'outback', 2015)
print(my_used_car.get_descriptive_name())
my_used_car.increment_odometer(10000)
my_used_car.read_odometer()
```



Classes 2 (using)

Um objeto do tipo Car é criado com marca audi, modelo a4 e ano 2019.

O Python utiliza o método `__init__(self, make, model, year)` para instanciar esse objeto.

A instância criada é associada à variável `my_new_car`.

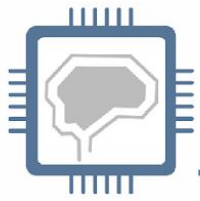
```
from car import Car
```

```
my_new_car = Car('audi', 'a4', 2019)
print(my_new_car.get_descriptive_name())
my_new_car.read_odometer()
```

```
my_new_car.odometer_reading = 23
my_new_car.read_odometer()
```

```
my_new_car.update_odometer(24)
my_new_car.read_odometer()
print()
```

```
my_used_car = Car('subaru', 'outback', 2015)
print(my_used_car.get_descriptive_name())
my_used_car.increment_odometer(10000)
my_used_car.read_odometer()
```



Classes 2 (using)

Para gerar uma descrição para o carro, utilizamos o método

`my_new_car.get_descriptive_name()`

A função `print` imprime a descrição.

O método `my_new_car.read_odometer()` imprime na tela uma mensagem contendo o valor do atributo `odometer_reading`

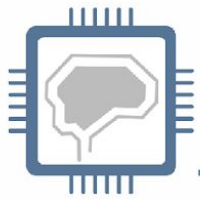
```
from car import Car
```

```
my_new_car = Car('audi', 'a4', 2019)
{ print(my_new_car.get_descriptive_name())
  my_new_car.read_odometer() }
```

```
my_new_car.odometer_reading = 23
my_new_car.read_odometer()
```

```
my_new_car.update_odometer(24)
my_new_car.read_odometer()
print()
```

```
my_used_car = Car('subaru', 'outback', 2015)
print(my_used_car.get_descriptive_name())
my_used_car.increment_odometer(10000)
my_used_car.read_odometer()
```



Classes 2 (using)

`my_new_car.odometer_reading` acessa diretamente o atributo `odometer_reading` e atribui o valor 23 a ele.

O método `my_new_car.read_odometer()` imprime na tela uma mensagem contendo o valor do atributo `odometer_reading`

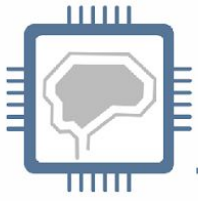
```
from car import Car
```

```
my_new_car = Car('audi', 'a4', 2019)
print(my_new_car.get_descriptive_name())
my_new_car.read_odometer()
```

```
{ my_new_car.odometer_reading = 23
  my_new_car.read_odometer()
```

```
my_new_car.update_odometer(24)
my_new_car.read_odometer()
print()
```

```
my_used_car = Car('subaru', 'outback', 2015)
print(my_used_car.get_descriptive_name())
my_used_car.increment_odometer(10000)
my_used_car.read_odometer()
```



Classes 2 (using)

O método

`my_new_car.update_odometer(24)` acessa o atributo `odometer_reading` e atribui o valor 24 a ele.

Usar métodos para ler e atualizar valores de atributos de uma classe é uma boa prática de programação.

O método `my_new_car.read_odometer()` imprime na tela uma mensagem contendo o valor do atributo `odometer_reading`

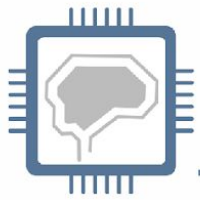
```
from car import Car
```

```
my_new_car = Car('audi', 'a4', 2019)
print(my_new_car.get_descriptive_name())
my_new_car.read_odometer()
```

```
my_new_car.odometer_reading = 23
my_new_car.read_odometer()
```

```
my_new_car.update_odometer(24)
my_new_car.read_odometer()
print()
```

```
my_used_car = Car('subaru', 'outback', 2015)
print(my_used_car.get_descriptive_name())
my_used_car.increment_odometer(10000)
my_used_car.read_odometer()
```



Classes 2 (using)

Um objeto do tipo Car é criado com marca subaru, modelo outback e ano 2015.

my_new_car.get_descriptive_name()

gera uma descrição do carro que é impressa pela função print.

my_used_car.increment_odometer(10000)

incrementa o valor do atributo

odometer_reading em **10000** unidades.

my_new_car.read_odometer() imprime na tela uma mensagem contendo o valor do atributo **odometer_reading**

```
from car import Car
```

```
my_new_car = Car('audi', 'a4', 2019)
print(my_new_car.get_descriptive_name())
my_new_car.read_odometer()
```

```
my_new_car.odometer_reading = 23
my_new_car.read_odometer()
```

```
my_new_car.update_odometer(24)
my_new_car.read_odometer()
print()
```

```
my_used_car = Car('subaru', 'outback', 2015)
print(my_used_car.get_descriptive_name())
my_used_car.increment_odometer(10000)
my_used_car.read_odometer()
```

```
from car import Car
```

```
my_new_car = Car('audi', 'a4', 2019)
print(my_new_car.get_descriptive_name())
my_new_car.read_odometer()
```

```
my_new_car.odometer_reading = 23
my_new_car.read_odometer()
```

```
my_new_car.update_odometer(24)
my_new_car.read_odometer()
print()
```

```
my_used_car = Car('subaru', 'outback', 2015)
print(my_used_car.get_descriptive_name())
my_used_car.increment_odometer(10000)
my_used_car.read_odometer()
```

Classes 2 (using)

Saída do programa

2019 Audi A4

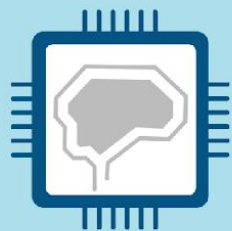
Esse carro rodou 0 mi.

Esse carro rodou 23 mi.

Esse carro rodou 24 mi.

2015 Subaru Outback

Esse carro rodou 10000 mi.



Inteligência Artificial e Computacional

Profa.: Sabrina de A. Silveira